

Untrusted-Actor Isolation: Current Gaps in the PVM

A findings register for the PVM team — what is reachable today, what closes it, and what does not

2026-07-31

Contents

1. What the barrier does today	2
2. Fix now — actor-triggerable validator compromise	3
3. The coverage gap	6
4. The findings in detail	11
5. Triage — what actually closes each of these	17
6. Verification and provenance	20
7. Open questions for the PVM team	26
Appendix A — Channel inventory	26
Appendix B — Ready-to-file tickets	31

What this is. The complete inventory of known actor-isolation gaps in the PVM, with the mechanism and a source anchor for each, triaged into what to fix now, what is worth doing as a bridge, and what no per-channel patch will close. It is meant to be checked, argued with, and burned down.

What this is not. It is not the architecture proposal. That document — `pvm-interpreter-isolation-proposal.pdf` — asks for a decision about where actor code should run. This one only describes what is true today. §5 is the one place the two meet, and it says so plainly.

Pin. Everything here is verified against `cowboyinc/node origin/devnet 2b2e73ab`. Every anchor resolves with `git show 2b2e73ab:<path>` **except the two that are explicitly branch-qualified at the point of use** — `pvm-runtime/src/lib.rs:1154` exists only at PR #1106 head `eeeb0b5f`, and `differential_ci.rs` exists only at PR #1105 head `4f96ded6` and not on devnet at all. Anchors are given as `<path-suffix>:<line>` — enough path to disambiguate, because several of these basenames are ambiguous in the tree (`lib.rs` alone matches 35 paths).

Confidence. Three labels, used consistently:

- **source-verified** — the code path was read at `2b2e73ab` and the anchors below are it.
- **runtime-reproduced** — it was executed and observed, with the configuration stated.
- **unassessed** — reachable on inspection, not yet exercised. Not counted as live.

Circulation. Engineering and security need-to-know while the `validator-halt` (§2.1) and the `root command-execution path` (§2.3) remain open.

1. What the barrier does today

COW-2434 and the guard around it are a real control, and the coverage discussion in §3 only makes sense once it is clear what they actually block. Five mechanisms are in place (the fifth surfaced only under live testing).

The object freeze and write barrier make baseline Python objects immutable to actor code. `check_frozen_write` (`vm/src/sandbox.rs:104-119`) raises on any mutation of a frozen object while a barrier scope is active. It genuinely blocks what it was built to block — verified controls include `copyreg.dispatch_table`, `warnings.filters`, `__bases__`, module-attribute writes through the Python `__setattr__/__setitem__` path, `sys.modules` writes, `builtins.__import__`, `__globals__`, and `contextvars`.

The deploy-time AST scan (`ACTOR_DENIED_IO_IMPORTS`, `execution/src/pvm_executor.rs:414-427`) rejects actor code that directly imports the dangerous high-level modules: `socket`, `subprocess`, `select`, `signal`, `threading`, `ssl`, `inspect`, and `_sre`.

The guard preamble (`pvm-runtime/src/guard.rs`) replaces `builtins.open`, `io.open`, and `builtins.execfile` with `_blocked_open` (`guard.rs:184,187,192`), a stub that raises `DeterministicValidationError`. So `open("/etc/passwd")` is blocked.

One precision that matters for §4.4: line 186 reads `import io as _io`, so the name `_io` there is a **local alias for the Python wrapper module `io`** — the rebinding on line 187 lands on `io.open`. The *native* module also called `_io` is a different object and is not touched.

The import allowlist — the strongest of the four, and the one most easily overlooked because it lives in the same preamble. `_pvm_import` replaces `builtins.__import__` (`pvm-runtime/src/guard.rs:149-164`) and admits a module only if `_is_allowed()` says so; that function checks `_DENY` first, then `_ALLOW`, then the per-call `_LIB_ALLOW`, then prefix matches, and **falls through to return `False`** (`pvm-runtime/src/guard.rs:68-97`). Imports are therefore **default-deny against a positive allowlist**, not a blocklist over an open environment. A `_PvmImportGuard` meta-path hook backs it up for **uncached** routes that still traverse `sys.meta_path` (`pvm-runtime/src/guard.rs:167`) — it is not a general backstop, because a cached module is returned before finder resolution ever happens (§3).

The deploy-time import resolver (fifth control, learned by on-chain test rather than source reading). At deploy the chain resolves every import in the actor’s source and **rejects any it cannot resolve to module source** with “*unresolved import*”. This blocks every *direct* native import — `import posix`, `import _io`, `import sys`, `import marshal` all fail to deploy, verified on-chain — because native builtins have no `.py`. It does not, however, reach a native capability held as an *attribute* of an allowlisted Python-source module (§2.3): `import io` deploys, and `io._io` comes with it.

That last control matters for reading §3 correctly. The problem there is not that an allowlist is missing — it is what the allowlist admits, and what remains reachable *inside* an admitted module.

These five do what they were designed to do. Everything in §§3–4 is **coverage** — surface the design did not set out to cover — not a defect in any of the five.

One change since this register's previous revision is worth calling out, because it is the pattern in miniature and the team found it independently. `ACTOR_DENIED_IO_IMPORTS` gained `"_sre"` (TOB-COWBOY-32) after 2026-07-27, and the comment added with it explains why the entry is deploy-scan-only rather than runtime-banned:

```
re (whitelisted) imports _sre internally at runtime, so banning it in the runtime
stdlib_blacklist would break re. This only denies actor code from directly importing _sre
at deploy time.
```

That reasoning is correct. It also means the deploy scan and the runtime blacklist now cover deliberately different sets — which is exactly the seam §3 is about, and the reason a name-based approach gets harder to keep sound as it grows rather than easier.

2. Fix now — actor-triggerable validator compromise

Three findings here let a deployed actor halt the validator (§2.1) or **run arbitrary commands as root inside it** (§2.3). A fourth mechanism (§2.2) is a source-verified process-kill that, when tested on-chain, did **not** reproduce — it is kept here, downgraded, because the reachability is real and the non-firing is worth understanding. None depends on how the isolation question is answered, and none should wait for it. §2.3 is confirmed live on-chain — a deployed actor executed `id` (returned `uid=0(root)`) and read `/etc/shadow` on a live validator.

2.1 `rustpython_checkpoint` reaches `std::process::exit(0)` — COW-2731

Mechanism. Any actor halts the validator:

```
import rustpython_checkpoint
def main(payload=None):
    rustpython_checkpoint.checkpoint_bytes()    # -> std::process::exit(0)
```

`rustpython_checkpoint.checkpoint("/path")` additionally performs an `fs::write` to an actor-controlled path, as the node uid, before exiting. Nothing catches it: this is a bare exit, not a panic, so the panic boundary discussed in §4.6 is not involved.

Reachability chain — source-verified. The module is actor-whitelisted (`pvm-runtime/src/determinism.rs:218`) and registered unconditionally (`vm/src/stdlib/mod.rs:99`). `checkpoint()` and `checkpoint_bytes()` arm `vm.state.checkpoint_request` (`vm/src/stdlib/rustpython_checkpoint.rs:8-33`). The trigger `maybe_checkpoint_request` is checked every instruction (`vm/src/frame.rs:516`) and **does not consult continuation mode** (`vm/src/frame.rs:2809-2827`); on trigger it reaches `if checkpoint_exit { std::process::exit(0); }` (`vm/src/frame.rs:595`).

`checkpoint_exit` **defaults true** (`vm/src/vm/setting.rs:227`) and is forced **false only** when continuation options are present (`pvm-runtime/src/lib.rs:576-579`) — which production never sets, because `message_id` is always `None` and so `continuation: None` is passed (`execution/src/pvm_executor.rs:1358`). The always-`None` `message_id` therefore gates the kill **open**, not closed.

Runtime-reproduced with production-config options (continuation: None), and **confirmed on-chain**: a deployed actor calling `checkpoint_bytes()` halted the validator on river (node `v0.0.34-0efc59f` = devnet HEAD), RestartCount `0→1`. COW-2731 is **In Progress**, not merged to devnet — the kill fires at HEAD.

The trap that masks it. A local harness built on `ExecutionOptions::default()` looks safe, because the default sets `continuation: Some(...)` (`pvm-runtime/src/lib.rs:286`), which forces `checkpoint_exit = false`. Production passes None. Anyone who tried to reproduce this with default options and saw nothing happen was reading a real result from the wrong configuration.

Fix. PR #1110 is open and not merged. It gates both the `checkpoint*()` pyfunctions and `maybe_checkpoint_request` on continuation/checkpoint mode, keeping the module functional for the continuation feature’s own tests (`checkpoint_gate.rs` 4/4; the continuation suite green).

Residuals stay open behind it, all latent today only because production never enters Checkpoint mode: the reflection-guard skip and the import-whitelist widening to `os/posix/importlib` require an actual resume, while pooling-off and the COW-2434 barrier bypass fire under Checkpoint *mode* even without one.

2.2 `_signal.raise_signal(SIGKILL)` — reachable, but does NOT kill the validator on-chain

This is a correction, and a caution against shipping source-verified kills unverified.

The mechanism is real on inspection: `_signal` is registered (`vm/src/stdlib/mod.rs:100`), exports `SIGKILL` (`vm/src/stdlib/signal.rs:119`), and `raise_signal()` (`vm/src/stdlib/signal.rs:449`) calls `libc::raise(signalnum)` (`vm/src/stdlib/signal.rs:470`); `_signal` is reachable from actor code via a nested import (confirmed on-chain: `SIGKILL=9`, `raise_signal` present). An earlier draft called this a validator kill on that basis.

On-chain it does not fire. A deployed actor that ran `import _signal; _signal.raise_signal(_signal.SIGKILL)` **survived the call** — the handler wrote a marker *after* the raise (`survived-signal`) and the validator’s RestartCount was unchanged (node `v0.0.34-0efc59f`). So the `SIGKILL` does not terminate the validator process from an actor frame; why (thread targeting, signal masking, or a stub in this build) is an open question for §7. The checkpoint kill (§2.1) is the confirmed process-halt; this one is downgraded to a reachable native call with no demonstrated kill. It stays in the register because the *reachability* of `_signal` is real (§3) and because a claimed kill that does not reproduce is exactly what on-chain testing exists to catch.

2.3 Actor-triggerable root command execution on the validator — CONFIRMED on-chain

A deployed actor can execute arbitrary shell commands **as root** inside the validator process. This is not inferred from source — it was deployed and executed as an ordinary actor on a live single-validator network (river, node `v0.0.34-b5b407d`). The actor ran `id` (returned `uid=0(root)`), `read / etc/shadow`, and its output was captured from the validator container; the validator runs as root, PID 1. This is the most serious finding in the register.

```
def main(payload=None):  
    import posix                                     # NESTED import — see below
```

```

    posix.system("id > /tmp/x; ...")          # runs as root in the
validator
    return b"ok"

```

Two facts compose, each already known to the audit.

Fact one — *posix* is *os* under a name nobody denied, reachable by the *import* statement. *os* is black-listed, deny-rooted, and cache-poisoned; *posix* (its Unix name, `vm/src/stdlib/posix.rs:23`) is preloaded into `sys.modules` at *init* (`vm/src/import.rs:38`) and appears on no list. An *import* *posix* statement compiles to the `import_inner` path, which passes the deny check and returns the **cache hit** (`vm/src/vm/mod.rs:851-860`) before `builtins.__import__` — so the positive allowlist, which would reject it, is never consulted (§3). *posix* carries `system`, `open`, `read`, `write` via the shared `_os` module (`vm/src/stdlib/os.rs:147, :1243-1245`). The audit’s recommendation for exactly this — “move deny-list enforcement ahead of the `sys.modules` fast path... cache poisoning is safe only when paired with VM-level policy enforcement” — is still open at this pin.

Fact two — *a nested import evades the resolver, and the deny-scan does not deny posix*. The deploy has two import checks. The **resolver** (`extract_top_level_imports`) inspects the actor’s **top-level** source only; the **deny-scan** (`find_denied_import`) walks the **full AST** (`walk_stmt`). A top-level import *posix* is rejected by the resolver (“unresolved import”, verified on-chain). A *nested* import *posix* in the handler body evades the resolver — and although the full-AST deny-scan does see it, it does not reject it, because *posix* is on no deny list. So it deploys. This is the mechanism of Trail of Bits finding **TOB-COWBOY-1 (COW-2418, marked Done)** — the auditors’ own exploit is a handler-body `import _posixsubprocess → fork_exec(['/bin/sh', '-c', ...])`. COW-2418 made the deny-scan walk the full AST *and* blocklists *os* and *subprocess*, so the nested-import class is closed **for the denied names**. But the list omits *posix* — the module that is *os* on Unix — and `_posixsubprocess`, the module the auditors used. So nested import *posix*; `posix.system(...)` reproduces the finding past its own fix. This is a live bypass of a closed critical ToB finding, not a new class.

Precise scope of what does and does not work (all tested on-chain):

- `import posix` **nested** in the handler → `posix.system` → **root command execution**. Confirmed.
- `import posix` **top-level** → deploy-rejected (“unresolved import”). The resolver catches the static form; the nested form is the bypass.
- `__import__("posix")` (explicit dynamic call) → runtime guard `_pvm_import` rejects it — the *explicit* call goes through the allowlist, unlike the import statement’s cache path.
- `import io; io._io.open(...)` → **host file read/write** via the allowlisted *io* wrapper’s retained native reference (§4.4). Confirmed on-chain independently (`read /etc/passwd`).
- `import _posixsubprocess` (nested) → the module the auditors’ own TOB-1 exploit used is reachable, with `fork_exec` present (confirmed on-chain). Its `fork_exec` payload was not re-fired, but `posix.system` above already demonstrates the command-execution class.
- `import builtins; builtins.exec(...)` → real `exec/compile` recovered (COW-2228, whose regression module `reacquisition_bypass_remains_open` is ignored/open). Deploys and runs; its inner `__import__` is still guard-checked.

Root cause, and why it is a class. Independently-sound controls compose with gaps: the deploy *resolver* sees only top-level source (so a nested import is not resolved); the deploy *deny-scan* walks the full AST but its list omits `posix/_posixsubprocess`; the runtime deny/allowlist is skipped by the import-statement cache path; and native capabilities sit under names (`posix`) the lists never covered. Freezing the Python policy (COW-2434) does not reach any of these — the auditors said so explicitly under TOB-16 (“making the lists immutable would only address this specific mutation primitive”).

Fix (ticket B4b). In the near term, the audit’s own recommendations, all still open: (1) make `validate_actor_code` reject nested *and* dynamic imports whose root is denied or native, and add `posix/_posixsubprocess` to the deny list COW-2418 already enforces recursively (reopen COW-2418); (2) enforce the allowlist at the VM level, ahead of the `sys.modules` cache, not only on a cache miss (the audit’s cache recommendation, and §3); (3) neutralize `posix/os` reachability and the `io` wrapper’s native references (§4.4). The durable answer is the process boundary of §5: under it, a child that runs `posix.system` runs it in a namespace with no host to reach, and the finding class closes by construction rather than by a longer deny list. —

3. The coverage gap

Module imports are default-deny by construction — with two holes in the enforcement, not just the policy. `_is_allowed()` (`pvm-runtime/src/guard.rs:68-97`) checks `_DENY`, then `_ALLOW`, then `_LIB_ALLOW`, then prefix matches, and falls through to return `False`; `_pvm_import` raises `NonDeterministicError` for anything that fails it (`pvm-runtime/src/guard.rs:149-161`). So the import gate is a positive allowlist, and this section is **not** the claim that one is missing.

The gap is in three places in the policy, plus one in the enforcement itself (the allowlist is a mutable set an off-barrier thread can edit, below).

First, what the allowlist admits. `_signal` is reachable not because imports default to allowed but because `_signal` is *on the list* — along with `_socket`, `_posixsubprocess`, `_ssl`, `fcntl`, `sys`, `io`, `math`, and `_thread`. §3.2 walks that through. The fix there is to shrink the list, which is bounded work on a list we control.

Second, the allowlist is only consulted on a cache miss. `import X` enters `VirtualMachine::import_inner` (`vm/src/vm/mod.rs:821`), which calls `check_import_allowed` (`vm/src/vm/mod.rs:654-675`) — a **deny-root** check, deliberately placed ahead of the cache by COW-2433 — and then looks up `sys.modules`. On a hit it returns the module immediately (`vm/src/vm/mod.rs:851-860`); only on a miss does it call `builtins.__import__` (:862-878), which is where `_pvm_import` and the allowlist live. So a module already resident in `sys.modules` **whose root is not a Rust deny root** is reachable without the allowlist ever being consulted. (A resident module whose root *is* a deny root, or is blacklist-poisoned, is still stopped — the deny check runs first. The reachable population is precisely the preloaded modules absent from every list.) §2.3 is that population, with `posix` → command execution as the lead and `_io` as the file-I/O instance.

There is also a way for actor code to change what the allowlist contains. The allowlist is a Python set (`_ALLOW`) reachable as `builtins.__import__.__globals__["_ALLOW"]`. The freeze barrier makes it immutable on the main actor thread, but a callback running on a thread spawned by

`_thread.start_new_thread` executes with the barrier’s thread-local flag defaulting off (§4.2), so it can `_ALLOW.add(name)` and make a previously-rejected module importable on the cache-miss path. Confirmed at runtime: a `colorsys` import rejected on the main thread succeeded after an off-barrier thread added it. The Rust deny roots still hold, so this widens the *allowlist*, not the deny set — but it means “default-deny against a positive allowlist” is only true for an actor that does not spawn a thread.

Third — and this is the part no name-based control reaches — there is no allowlist *inside* an admitted module. Once `sys` is importable, every attribute hanging off it is reachable: `orig_argv`, `_rustpython_debugbuild`, `settrace`, `set_int_max_str_digits`. Once `io` is importable — and it must be — `FileIO` arrives with it, which is why the guard could correctly replace `open` and still leave host file I/O reachable beside it (§4.4). The admitted-module set is enumerable; the attribute surface hanging off it is not, and it grows with each RustPython release rather than with anything we write.

That is the property Trail of Bits identified in finding 16, stated precisely: the cost of completeness is *per-type* within admitted modules, not *per-name* at the import boundary. It is not a bug in any of the five controls in §1; it is a statement about what a Python-level barrier can be asked to guarantee once any non-trivial module is on the list.

3.1 Five faces

The uncovered surface is not one bug. It has five distinct faces, each a set of anchors:

Face	What is reachable	Representative anchors
VM fields	sys.settrace/setprofile retain a callable in a mutable pub VM field that fires inside a later actor	vm/src/stdlib/sys.rs:796, vm/src/protocol/callable.rs:51,53, vm/src/vm/mod.rs:73-75
Stdlib state	~12 registries, thread-locals, and defaults the freeze walk does not traverse	vm/src/vm/thread.rs:12-14, vm/src/vm/mod.rs:106 (see §4.5)
_thread	barrier bypass via a spawned thread; get_ident() returns an unremapped thread id	vm/src/sandbox.rs:27, vm/src/stdlib/thread.rs:265,317-325
Native modules	_signal, _posixsubprocess, _socket, _ssl, fcntl — on the import allowlist, not blacklisted, and the underscore spellings are absent from the AST scan	pvm-runtime/src/determinism.rs:104, vm/src/stdlib/mod.rs:100, stdlib/src/lib.rs:159,168
Ambient environment	io.FileIO reopens the host file I/O the guard’s own open block closes; sys.orig_argv is an unpinned validator-local value	vm/src/stdlib/io.rs:4738, :4829, vm/src/stdlib/sys.rs:210-211

3.2 The native-module face is the whole argument in miniature

The consensus executor builds its DeterminismOptions **inline** (execution/src/pvm_executor.rs:1241-1354) rather than calling DeterminismOptions::actor_execution() (pvm-runtime/src/determinism.rs:62). At this pin the two carry the same whitelist and the same ten-name blacklist, so the reachability conclusions below hold either way — but the inline object is the one production actually uses, and it also carries height-dependent memory metering the named constructor does not. Three lists interact:

- **The runtime whitelist** admits the low-level modules *alongside* their high-level wrappers: socket/_socket, signal/_signal, subprocess/_posixsubprocess, ssl/_ssl, fcntl/_fcntl — though see the note below on _fcntl.
- **The runtime blacklist** (pvm-runtime/src/determinism.rs:128) denies exactly ten names: time, random, pickle, datetime, os, ctypes, _ctypes, cffi, _cffi_backend, cmath. **No native OS module is on it.**
- **The deploy AST scan** bans the high-level names only — socket, subprocess, select, signal, threading, ssl, inspect, _sre.

So a *nested* import _signal passes both deploy gates (the deny-scan does not list it and the resolver only sees top-level source), sits on the runtime whitelist, and is absent from the runtime

blacklist. The same holds for `_posixsubprocess`, `_socket`, and `_ssl`. (A *top-level* native import is instead rejected by the resolver as “unresolved”; the nested form is the one that deploys.)

One precision worth stating, because it cuts the other way from the obvious reading. The high-level name `socket` is denied by the **deploy deny-scan** (`ACTOR_DENIED_IO_IMPORTS`) but is **not** on the *runtime* blacklist — in production it sits on the whitelist (`_ALLOW`) and is absent from `_DENY`. So a dynamic `__import__("socket")` evades the deploy deny-scan (which reads static import statements) and, at runtime, is *admitted* by the guard (`_is_allowed` finds it in `_ALLOW`, and `check_import_allowed`’s deny roots do not include it). The deploy deny-scan is therefore the only control standing between an actor and `socket`, and a dynamic import removes it. (Source-reasoned; not separately exercised on-chain. Adding `socket` to the runtime `_DENY` would block both spellings.) The burden here is the alias set and the per-release growth of the module list, on top of this dynamic-import gap — the seam the `_sre` comment in §1 documents from the inside.

3.3 The census, and why it stops

More than **twenty identified channels** fall outside the barrier’s coverage across the five faces. (Appendix A lists forty-four numbered entries: the count here is *channels* an actor can reach, while the appendix additionally carries the liveness, atomicity, and freeze-bypass findings, which are defects rather than reachable channels.) At least **eleven** are live primitives on devnet today, all source-verified at 2b2e73ab; where a claim was additionally tested against a live chain the result is noted. The most serious confirmed escape is **root command execution** via nested `import posix` (§2.3):

Live channel	Class	Consequence
import posix; posix.system(...) (§2.3)	root command execution	arbitrary shell command as root — confirmed on-chain
rustpython_checkpoint.checkpoint_bytes() (§2.1)	validator-kill	halts the validator — confirmed on-chain (COW-2731 not fixed)
_signal.raise_signal(SIGKILL) (§2.2)	validator-kill	reachable, but does not kill on-chain (downgraded)
_posixsubprocess.fork_exec	host escape	fork/exec of an actor-selected binary — module reachable, fork_exec present
sys.settrace / setprofile	cross-actor code execution	callback fires inside a later actor
_thread.start_new_thread (barrier bypass)	cross-actor mutation	spawned thread mutates frozen baseline
_thread.get_ident	consensus input	unremapped validator-local thread id
_socket.gethostname	consensus input	validator box hostname
sys.orig_argv	consensus input	validator command line
sys._rustpython_debugbuild	consensus input	debug-vs-release build oracle
io._io.open / io.open_code / io.FileIO (§2.3)	host escape	host file read/write via the allowlisted io wrapper — confirmed on-chain

COW-2731's checkpoint kill (§2.1) is the confirmed validator-halt and heads the table above. `_ssl` and `fcntl` are **unassessed** members of the same native class — on the import allowlist and denied at neither gate, but not exercised.

One correction worth making explicitly, because it cuts against this document. The allowlist also contains the string `"fcntl"` (pvm-runtime/src/determinism.rs:105), but **no module of that name exists**: the Unix native module is registered as `"fcntl"` => `fcntl::make_module (stdlib/src/lib.rs:159)` and its `#[pymodule]` is named `fcntl`. `import _fcntl` therefore fails at module resolution. The reachable module is plain **`fcntl`** (pvm-runtime/src/determinism.rs:104), which is on the allowlist and on neither deny list. The finding stands; the name was wrong. A dead string on an allowlist is also its own small argument for shrinking the list — nobody noticed it was inert. They are deliberately not counted as confirmed live; they should be evaluated under the same capability review.

The enumeration stops here on purpose. The surface is the entire stock interpreter, so the list has no natural end, and continuing would only re-prove finding 16 at greater length. §5 is about what to do given that, rather than how much longer the list could get.

4. The findings in detail

Grouped by consequence. Each is source-verified at 2b2e73ab unless labelled otherwise.

4.1 Cross-actor code execution

sys.settrace — an actor’s callback runs inside a later actor. An actor calls `sys.settrace(f)`; the VM retains `f` in `vm.trace_func` (`vm/src/stdlib/sys.rs:796`, attribute at `:794`). On a later pooled execution — **a different actor** — `f` fires on frame events (`vm/src/protocol/callable.rs:51,53`) and can write that actor’s module globals. The pool key (`pvm-runtime/src/lib.rs:486-503`) carries **no actor address and no code hash**, so one actor’s callback can affect any subsequent actor on the same OS thread. `sys.setprofile` is the same mechanism (`vm/src/stdlib/sys.rs:769-772`).

Confidence: mechanism source-verified; runtime result via an uncommitted harness.

The reproduction exercised the production `PvmExecutor::execute_handler`, alternating two actor addresses (`C0DE` sets the trace, `BEEF` runs the injected callback). The committed #1105 corpus goes through the `pvm-runtime` path instead, so the runtime result is **not currently reproducible from the tree**. Closing that gap is a ticket in Appendix B.

Siblings within one transaction share an interpreter. Nested `call_actor` runs through the same pool (`execution/src/pvm_host.rs:3037,2086,2109`). The outer execution empties the pool slot for its `enter()` (`pvm-runtime/src/lib.rs:606-629`), but each nested `enter()` writes its interpreter back on completion (`pvm-runtime/src/lib.rs:703-715`). So callee A returns a warm interpreter that sibling callee B then runs on, inside a single transaction. A per-transaction reset does not reach this; the reuse is intra-transaction.

A side effect worth knowing: because the outer execution empties the slot, the **first** nested call in a transaction takes the pool-miss branch and builds a cold interpreter. §7 asks for the measurement that would tell us what that costs in production.

4.2 Cross-actor mutation — the `_thread` barrier bypass

COW-2434’s write barrier is gated on a thread-local. `check_frozen_write` returns `Ok(())` whenever `BARRIER_ACTIVE` is false (`vm/src/sandbox.rs:104-119`), and `BARRIER_ACTIVE` defaults false (`vm/src/sandbox.rs:27`), set only on the main actor thread via `BarrierScope::enter()` (`pvm-runtime/src/lib.rs:1156`).

A thread spawned by `_thread.start_new_thread` (`vm/src/stdlib/thread.rs:298-327`) **never enters BarrierScope**, so on that thread the barrier is off — while its `ThreadedVirtualMachine` shares the same frozen builtins, `sys`, `ctx`, and `PyGlobalState`. The spawned thread can therefore mutate a frozen baseline object that the main actor thread cannot, synchronize before returning, and leave the interpreter mutated for the next actor.

`_thread` is whitelisted (pvm-runtime/src/determinism.rs:194). The only thing standing against this today is a prose comment (execution/src/pvm_executor.rs:1269-1270) explaining why threading is permitted.

4.3 Consensus-divergence inputs

An actor that returns any of these forks the chain, because the value differs across validators or across a restart.

id() cold vs. warm. Identical actor source returns a different `id()` on a cold interpreter (`first=128`) than on a warm one (`first=1`), so a **restarted validator computes a different result from a long-running one**. Root cause: `ExecObjectIdScope` is installed (pvm-runtime/src/lib.rs:753) *before* the cold-only guard/freeze/preamble setup (pvm-runtime/src/lib.rs:761-791), so cold setup consumes identity-sequence entries that warm setup skips.

PR #1106 (open, head `eeeb0b5f`) calls `reset_exec_object_ids()` in `run_source` at the `BarrierScope` point where actor bytecode begins (pvm-runtime/src/lib.rs:1154 *on that branch*), after every cold-only preamble; the outer scope still owns `save/restore`, so nested `call_actor` is unaffected. Because it changes actor-visible `id()/repr()` values it is a **state-transition- function change**, and must ship in a coordinated genesis-replacement upgrade rather than an in-place patch.

Two caveats worth stating precisely rather than claiming blanket coverage:

- The continuation *resume* path (pvm-runtime/src/lib.rs:802-807) bypasses `run_source` and never receives the reset. This is fine, because *resume* is always cold and non-pooled (`resume_bytes` is `Some` only under `ContinuationMode::Checkpoint`, which forces `use_pool=false`), so there is no warm resume to diverge against.
- **Address reuse is not closed by #1106.** `EXEC_OBJECT_IDS` is keyed on the raw heap pointer (vm/src/object/det_id.rs:35,44-54); the reset fixes the counter *offset*, not pointer *reuse*. `a=C(); i=id(a); del a; b=C(); return i==id(b)` can still differ cold vs. warm, because a warm interpreter's allocator free-lists are fragmented differently. **Unassessed** — it may be unreachable in practice — but a flat “fixed” would invite exactly that probe.

_thread.get_ident. Returns a hash of the raw `std::thread::current().id()` (vm/src/stdlib/thread.rs:265) — a per-process, validator-local Rust `ThreadId` that does not pass through the `EXEC_OBJECT_IDS` remap, which covers object `get_id()` and not thread ids. It is directly returnable, so two validators on different exec threads compute different results. `start_new_thread` likewise returns the spawned thread's local id (vm/src/stdlib/thread.rs:317-325).

This one **needs no thread spawn**, so it survives a “deny thread creation” fix. Gating the whole actor-visible `_thread` API is what closes it: `get_ident`, `start_new_thread`'s return value, `timed locks`, `_count`, `interrupt_main`, and `stacksize`.

sys.orig_argv. The runtime pins `sys.argv` to a deterministic value (vm/src/stdlib/sys.rs:127), but `sys.orig_argv` returns raw `env::args()` (vm/src/stdlib/sys.rs:210-211) — the validator's actual node-binary command line, including flags and config paths — and the guard does not pin it. (`sys._baserepl()` is a further reachable blocking `stdin` read.)

`_socket.gethostname`. Returns the box’s real libc hostname (`stdlib/src/socket.rs:2584`) — a consensus fork with no network involved at all.

The `sys` module carries build- and platform-identity attributes, and is denied at neither gate. This one is worth reading carefully, because the configuration looks safe and is not. `sys` is on `default_blacklist()` (`pvm-runtime/src/determinism.rs:236`) — but production `actor_execution()` does not extend that list, it **replaces** it with its own ten names (`pvm-runtime/src/determinism.rs:128`), and `sys` is not among them. `sys` is simultaneously on the production whitelist. So in production it is reachable, and three attributes on it are environment identity:

- **`sys._rustpython_debugbuild`** (`vm/src/stdlib/sys.rs:52`) is literally `cfg!(debug_assertions)`. An actor that returns it makes a debug-built node and a release-built node disagree — a build-profile oracle, readable in one attribute access.
- **`sys.platform`** (`vm/src/stdlib/sys.rs:70`) and **`sys.maxsize`** (`vm/src/stdlib/sys.rs:66`).
- **The target triple**, baked in at build time as `RUST_MULTIARCH = env!("RUSTPYTHON_TARGET_TRIPLE")` (`vm/src/stdlib/sys.rs:43`) and reachable through `multiarch()` (`vm/src/stdlib/sys.rs:47-48`).

This is the same shape as the checkpoint trap in §2.1: a default configuration that denies the module, a production configuration that does not, and no test that runs the production one.

Floating point reaches the platform’s libm. `math` is on the actor whitelist (`pvm-runtime/src/determinism.rs:172`), and `float **` lowers to the host pow. RustPython’s `math` delegates to the `pymath` crate (`pvm/Cargo.toml:249`), which is a C-ABI shim over the system `math` library, so results are a property of the machine’s libm rather than of the protocol. Rust documents its own transcendental under “unspecified precision”, which varies by platform and version.

Confidence: source-verified for reachability, unassessed for impact. That `math` is reachable and that `float **` lowers to the host is verified here. Whether any specific operation actually produces different bits across the machines we run — and which operations — has not been measured, and `pymath`’s internals are outside this tree. It is listed because a consensus-input class with no finding against it is worse than one with an honest “not yet measured”, and because the remedy is a decision rather than a patch: deny `math` and `float **` to actors, or build only for a target whose `math` path is pure Rust.

4.4 Escapes to the host

`_posixsubprocess.fork_exec`. Registered on Unix (`stdlib/src/lib.rs:193`), calls `nix::unistd::fork()` (`stdlib/src/posixsubprocess.rs:35,48`), and the child reaches `libc::execve` on an actor-selected path (`stdlib/src/posixsubprocess.rs:380`) — a real process spawn surface even before a successful `exec`.

`_socket network egress`. `_socket.socket()` builds a real `socket2::Socket` (`stdlib/src/socket.rs:1533`) and `connect()` is real kernel egress (`stdlib/src/socket.rs:1319`). `_socket` is registered on the production non-Wasm target (`stdlib/src/lib.rs:168`).

`io.FileIO`. The guard replaces `builtins.open`, `io.open`, and `builtins.execfile`, but `io.FileIO` (`vm/src/stdlib/io.rs:4738`) reaches `crt_fd::open` (`vm/src/stdlib/io.rs:4829`) along a path

beside the block the guard correctly installs. This is an illustration of why the completeness cost is *per-type* rather than *per-name*: the guard closed `open`, and the type that also opens files remained.

Native `_io` host file I/O. `_io` is the file-I/O instance of the import-boundary class in §2.3: `import _io; _io.open("/etc/passwd")` reaches native `_io.open` (vm/src/stdlib/io.rs:4387-4393) by the same cache-hit path as `posix`, because the guard’s file-I/O block rebinds `io.open` on the Python wrapper and not the native module (§1). **Runtime- reproduced** — read `/etc/passwd` and wrote a temporary file through native `_io` in the production-determinism harness.

The wrapper is not a safe path either. Even without importing `_io`, the *allowlisted* Python wrapper `io` reaches the same native code, so the cache bypass is not required:

- `pvm/Lib/io.py:52` does `import _io` and retains it as the module attribute `io._io`, so `io._io.open(path)` reaches native `_io.open` through an affirmatively permitted import.
- `pvm/Lib/io.py:55-56` copies `open_code` and `FileIO` out of native `_io` into `io`; the guard rebinds `io.open` but not `io.open_code`, so `io.open_code(path)` is live (vm/src/stdlib/io.rs:4397-4399, which calls `io_open(..., "rb", ...)`).

This matters for the fix: moving the allowlist ahead of the cache closes `import _io` but **not** `io._io.open` or `io.open_code`, because the wrapper already holds the native references. Those copied/back-referenced attributes must be neutralized directly, or reached only under the process boundary. Ticket B4b covers both.

Confidence: source-verified for the wrapper routes, runtime-reproduced for native `_io.open` and `posix.system`. The evidence base for §2.3 and this section is the production-determinism harness, not a live node — with the exception of the river deployment noted in §2.3.

4.5 The leak-channel floor

COW-2434 freezes Python *objects*. These channels reach Rust `thread_local!`s, `PyGlobalState` registries, and objects at which the freeze walk stops.

Where it stops is explicit: `freeze_module_structure` (vm/src/sandbox.rs:328) descends into four container types — `PyDict`, `PyList`, `PySet`, `PyByteArray` (vm/src/sandbox.rs:382-385) — and five structural types — `PyModule`, `PyType`, `PyFunction`, `PyNativeFunction`, `PyBoundMethod` (:397-401). Anything that is neither hits `continue` (:402-403). Tuples, property descriptors, `WeakSets`, and ordinary instances are all in that “neither” set, which is why the rows below are reachable rather than a defect in the walk. Probe-observed unless marked source-verified.

Channel	Mechanism
<code>sys.set_asyncgen_hooks</code>	Rust TLS (vm/src/vm/thread.rs:13-14); attaches to the next actor's async generators
<code>sys.set_coroutine_origin_tracking_depth</code>	Rust TLS (vm/src/vm/thread.rs:12)
<code>_socket.setdefaulttimeout</code>	process-global AtomicCell; whitelisted, on neither deny list
<code>atexit.register</code>	vm.state.atexit_funcs, no per-execution reset
<code>codecs.register / register_error</code>	native codec registry
<code>copy.deepcopy_nil=[]</code> default	the defaults tuple is not a walked container (vm/src/sandbox.rs:382-385)
closure-cell replacement (typing wrapper)	cells are skipped deliberately (vm/src/sandbox.rs:373-375), on the reasoning that module-__dict__ paths reach what must be immutable and closure-local caches are reset per execution; the residual is a cell reached by neither route
<code>_abc_registry WeakSet</code>	not one of the four walked container types (vm/src/sandbox.rs:382-385)
property descriptor	not structural, not a container, so the walk stops (vm/src/sandbox.rs:397-403)
<code>_thread._set_sentinel</code>	appends PyRef<Lock> to SENTINELS (vm/src/stdlib/thread.rs:360-367), cleared only at thread exit (:341) which the pooled thread never reaches → unbounded
<code>sys.set_int_max_str_digits</code> (<i>source-verified</i>)	AtomicCell<usize> (vm/src/vm/mod.rs:106), set via vm/src/stdlib/sys.rs:752-755 (accepts 0), read on every int↔str conversion, reset nowhere — contrast recursion_limit, re-pinned per execution (pvm-runtime/src/lib.rs:758-760)
lazy-import sys.modules persistence (<i>source-verified</i>)	the first import of a not-yet-loaded whitelisted module freezes the new module but leaves the cache entry (vm/src/vm/mod.rs:821-890); a later actor reads a changed len(sys.modules)

Bytecode caching is on by default. write_bytecode defaults to true (vm/src/vm/setting.rs:240) and nothing in pvm-runtime/ or execution/ overrides it, so the interpreter is

configured to write .pyc files and to prefer a matching one over recompiling. Whether it can fire depends on whether any import path is writable to the node uid at runtime, which has not been tested — **unassessed**, and listed here because the safe setting is one line and the failure mode if it does fire is that a cached artifact, not the deployed source, decides what executes.

One related bypass. The Rust-level `PyDict::set_item` used by `StoreGlobal` skips the `writable()` check that the Python `__setitem__` enforces (`vm/src/builtins/dict.rs:645-655` versus `:274-283`, where `self.writable(vm)?` sits at `:281`). So a module-global assignment from *within* stdlib Python — for example `threading.settrace()` writing `_trace_hook` (`Lib/threading.py:72-80`) — bypasses the freeze. This is why neutralizing `sys.settrace` alone does not close the `settrace` class.

4.6 Two system-level liveness facts

A panic in actor execution halts the validator. `catch_unwind` is not absent — `rpc/src/rpc.rs:747` supervises the RPC listener (COW-2709). What holds is narrower and worse: there is **no catch boundary around actor execution or commit** in execution, chain, or validator, and `no_panic = "abort"`. `validator/src/main.rs:342` sets `.with_catch_panics(false)`, so an uncaught task panic `resume_unwinds` the whole process, consensus included (`rpc/src/rpc.rs:367-376`). The only `catch_unwind` on the execution/RPC path is `rpc/src/rpc.rs:747`; a second at `codegen/src/symboltable.rs:282` is in the vendored compiler and off the execution path.

The consequence is that the real severity of ToB findings 5, 6, and 8 is not “unwinds with no rollback” — it is **a panic in actor execution halts the validator**. They are liveness bugs.

Metering is wired but not enforcing — the mesa halt. COW-2419 wired per-instruction gas into `cycles_used` (`execution/src/execution/transaction.rs:616-619`), but `pvm_instr_gas_price_ppm` defaults to 0 (`execution/src/basefee.rs:119,244-249`), so PVM CPU currently costs nothing. More importantly it folds at fee-computation time *after* execution — `vm/src/frame.rs:455-467` only saturating adds, with **no in-loop limit check** — so it can never abort a CPU-heavy handler mid-run.

This is the mesa-halt mechanism of 2026-07-14: two game-actor timer handlers each ran roughly 500–900 ms of wall clock and, coinciding, pushed proposal just past the 1000 ms `LEADER_TIMEOUT` to about 1.02 s, with `gas ≈ 0`. Enabling the price would not have prevented it; only **in-loop metering with a mid-run abort** does. The plumbing COW-2419 built is exactly what is needed — the remaining step is enforcing it inside the loop, which is a completion rather than a rebuild.

4.7 Host paths that write the store and bypass rollback()

Distinct from everything above: this is an **atomicity** gap rather than an isolation one, and it is reachable without any adversarial actor. It is included because it has the same shape — a control that is correct for what it covers, with surface outside it — and because it is a correctness issue for the PVM today.

`rollback()` (`execution/src/pvm_host.rs:1304-1328`) clears the staged in-memory state: `storage_cache`, events, outgoing messages, the three timer collections, deferred transaction

requests, the token request and pending queues, the sync-call maps, `account_balance_deltas`, and the call-stack fields. It cannot undo a write that has already reached the store.

At least ten host paths write the store during execution and bypass it. This is a floor, not a count — it came from reading `pvm_executor.rs` and `pvm_host.rs`, not from a mechanical check. The known set: the CIP-29 subscription mutators (`subscribe`, `unsubscribe`, `force_unsubscribe`, `update_bid`, `topup`), `acquire_epoch_access`, `register_account_key`, the stream-key-manager branch of `call_actor`, `emit_event`’s sync-fire, and `econ_intent_gate`.

`econ_intent_gate` (`execution/src/pvm_host.rs:1520-1600`) documents the behaviour in its own comment (:1542):

Direct store I/O mirrors the settlement ledger’s model (batch-committed, no per-tx rollback
→ a failed tx over-counts, which is fail-closed for a rate limit).

That reasoning is sound *for a rate limit*, where over-counting is the safe direction. It does not generalize to the other nine paths, and nothing marks which of them share it. `acquire_epoch_access` additionally performs a native CBY `set_account` outside `account_balance_deltas` (`execution/src/stream_key_manager.rs:1067,1078`), so that write is outside the balance-delta accounting as well as outside rollback.

Why the effect list does not already cover this. `ExecutionSideEffects` (`execution/src/pvm_executor.rs:54-67`) carries only outgoing messages, three timer collections (`scheduled_timers`, `cancelled_timers`, `extended_timers`), and events. Storage writes live in `ctx.storage_cache`; the six token queues, the pending maps, `account_balance_deltas`, `code_bytes`, `actor.nonce`, and entitlement counters are all outside it. So there is no single structure whose contents define what a transaction committed.

An independent sweep reached the same place. A separate review of state-transition-function determinism, working from transactional-boundary violations rather than from the interpreter, landed on the same finding — that a failed transaction can retain pending writes — and traced it to the same design flaw Trail of Bits filed as “transaction execution lacks per-transaction atomicity”. Two lines of enquiry converging on one defect is worth more than either alone, and it is the reason this is in bucket 1 rather than filed as a curiosity.

A hand count is the wrong instrument here. The durable form is a mechanically-checked store-write choke point, so that “which paths write the store” becomes a property the compiler or a lint enforces rather than something a reader has to re-derive. That also converts the “at least ten” above into an exact number, which is the first thing worth knowing.

5. Triage — what actually closes each of these

Every finding above sits in exactly one bucket below, with the reason stated. The buckets differ in *kind*, not in priority order: the first two are work that stands on its own, the third is a question this register cannot answer by itself.

Bucket 1 — Fix now, regardless of any architecture decision

These are not contingent on anything. They should not wait.

1. **Checkpoint kill** — A1 (§2.1, COW-2731) — land PR #1110. *Open, not merged.*
2. **_signal reachability** — A2 (§2.2) — the SIGKILL does **not** kill on-chain (downgraded), but `_signal` is a reachable native and part of the class in item 5. Fold into the native-class default-deny rather than treating it as a standalone fix-now kill. *No PR.*
3. **id() cold/warm consensus divergence** — A10 (§4.3) — land PR #1106. *Open, not merged.*
4. **Store writes bypassing rollback()** — A39, A40 (§4.7) — add a mechanically-checked store-write choke point, converting “at least ten” into an exact set. *No PR.*
5. **Root command execution via nested import posix** — A42 (§2.3) — confirmed live on-chain (ran `id` as root, read `/etc/shadow`); plus host file read/write via the `io` wrapper A41. These are residuals of COW-2284 and COW-2418 (both Done) that key off the blacklist and miss `posix`; COW-2228 (builtins/exec) is Canceled. Ticket B4b covers the fixes (neutralize `io._io/io.open_code/io.FileIO`, key the cache mitigation off the allowlist). *No PR.* (Related: A43 `_ALLOW`-mutation bypass, harness-reproduced; A44 `builtins.exec`, confirmed. See §6.4.)

Item 4 differs from the first three in kind: it is structural work rather than a patch, and it is **not** superseded by any isolation boundary — a staged commit contract needs an authoritative answer to “what did this transaction write” under every architecture. Starting it early is useful regardless of how §5 bucket 3 is resolved.

Item 3 carries a deployment constraint the other two do not: it changes actor-visible `id()/repr()` values, so it is a state-transition-function change and must ship in a coordinated genesis-replacement upgrade, not an in-place patch. That makes it slower to land than its severity suggests, which is a reason to start it early rather than a reason to defer it.

Bucket 2 — Bridge controls: real risk reduction now, superseded rather than wasted later

Each of these closes a live class today. If the boundary in bucket 3 is eventually built, these become redundant — but they are not wasted, because they are what stands between now and then.

5. **Default-deny the native OS-module class** at runtime — `_signal`, `_posixsubprocess`, `_socket`, `_ssl`, `fcntl` and their high-level wrappers. *Closes A16, A19, A20, A21, A25 — §3.2, §4.3, §4.4, §4.5.*
6. **Shrink the import allowlist, and pin the attribute surface of what remains.** A default-deny import allowlist already exists (§3, `pvm-runtime/src/guard.rs:68-97`), so this is not construction — it is reducing an over-broad list and then dealing with what the survivors expose. Two halves, and the second is the one that is easy to skip: drop the entries that grant capability without need (and the inert ones — see `_fcntl`, §3.3); then pin or remove the attribute surface of modules that must stay, starting with the `sys` identity attributes (`orig_argv`, `_rustpython_debugbuild`, `platform`, `maxsize`, `target triple`), which are reachable in production because the executor’s inline config does not carry `sys` on its blacklist even though `default_blacklist()` does. *Closes A12, A13, A14, A18, A22, A28, A29 — §3.1, §3.3, and the sys rows of §4.3.*

7. **Gate the whole actor-visible `_thread` API** — deny creation *and* pin or deny `get_ident`, `start_new_thread`'s return, timed locks, `_count`, `interrupt_main`, `stacksize`. *Closes A7, A8, A9 — §4.2, §4.3, §4.5.*
8. **Neutralize `sys.settrace` / `setprofile`** — and treat the `StoreGlobal-into-a-baseline-module-global` path (`threading.settrace`) as a separate audit item, because it survives the direct fix. *Closes A3, A4, A36 — §4.1, §4.5.*
9. **Scoped `asyncio-TLS save/install/restore` per execution.** *Closes A26, A27 — the `asyncgen` and `coroutine-origin` rows of §4.5.*
10. **Decide the float story** — either deny `math` and `float **` to actors, or build only for a target whose `math` path does not go through the system `libm`. *Closes A17 — the `float` row of §4.3.* This one is a decision rather than a patch, and it is the only bucket-2 item that may need a measurement before it can be made.

Two notes on how these are specified, because the obvious version of each is subtly wrong:

- **Item 5 must be enforced at runtime, not by extending the AST blocklist.** The deploy scan reads import statements, so a dynamic `__import__` evades it, and the underscore aliases were never on it. It also has to cover modules already imported by the preamble, not just actor imports.
- **Item 9 is a `save/restore` rather than a denial.** Async-generator hooks and `coroutine-origin` tracking are load-bearing for `asyncio` (`Lib/asyncio/base_events.py:633-649`), so denying the setters outright would break a direct-`asyncio` actor. The on-chain SDK deliberately avoids `asyncio` — `Lib/cowboy_sdk/actor.py:486`, where the comment at :481-485 explains that importing `asyncio` eagerly evaluates nested typing annotations and makes every `async` actor undeployable (COW-2331) — so the exposure is a *direct-asyncio* actor rather than an SDK one.

Bucket 2 is not the strategy. It is the part of the strategy that can start on Monday. The next bucket is why it is not the whole of it.

Bucket 3 — What per-channel patching does not close

This bucket is A5, A6, A30, A31, A32, A33, A34, A37, A38 — nine entries in Appendix A for which no per-channel patch in buckets 1 or 2 is proposed.

Buckets 1 and 2 name roughly a dozen specific channels. §3.3 counts more than twenty identified, and says plainly that the number is a floor rather than a total, because the reachable surface is the entire stock interpreter. Every review round of this material has surfaced another one — `io.FileIO` (§4.4) was found *after* the guard had correctly closed open, along a path beside it.

That is the shape of the problem: the cost of completeness here is *per-type*, not *per-name*, and the type list is not ours — it is RustPython's, and it grows on their release schedule. A name-based control can be made correct on any given day. Keeping it correct is an ongoing obligation that scales with someone else's codebase.

The recommendation of the companion document is to stop paying that cost by moving the boundary. `pvm-interpreter-isolation-proposal.pdf` proposes a long-lived sandboxed worker process that forks a clean interpreter snapshot for each actor frame, so a child that is poisoned,

panics, or exhausts its budget dies alone and the next actor starts from an unmutated snapshot. The validator keeps authority over chain state and accepts only typed, validated effects from the child over a private channel.

The argument for it is that it makes the *class* unreachable rather than closing its current members: no interpreter is reused across actors, so a completeness proof over channels stops being required. Ask the PVM owner for that document if you want the design, cost, and rollout — none of which is repeated here.

What would be more useful than agreement. Neither document has found a way to make the Python-level barrier complete, and both concluded the same thing from that. That is two conclusions from one line of reasoning, not independent confirmation. If you can see a third option — a way to make the environment positively constructed that we have dismissed too quickly, a RustPython-level change that closes the type class at its root, a different isolation primitive — that is a better outcome than either document’s recommendation, and this register exists partly to give you the evidence to make that case.

Bucket 4 — Unassessed

This bucket is A11, A15, A23, A24, A35.

`_ssl` and `fcntl` are on the production import allowlist and denied by neither gate, so they are reachable on inspection. They have not been exercised, so they are **not** counted among the eleven live channels in §3.3. They should be evaluated under the same capability review as item 5.

`write_bytecode defaults true` (§4.5) and is not overridden, so the interpreter is configured to write and prefer `.pyc`. Whether it can fire depends on runtime filesystem writability, which has not been tested. The safe setting is one line either way.

Two further items carry the same label for the same reason: **`address reuse after the id()`** **fix** (§4.3), which may be unreachable in practice but is not demonstrated either way, and the `sys._baserepl()` blocking stdin read noted alongside `sys.orig_argv`.

6. Verification and provenance

Pin. `cowboyinc/node origin/devnet 2b2e73ab`, frozen for this document. Every anchor was re-verified at this revision, line by line, including each component of multi-line and comma-separated anchors — with the two branch-qualified exceptions named in the header and again below. `devnet` is an integration branch and moves daily; a stale-but-stated pin is checkable, a drifting one is not.

Origin. Trail of Bits comprehensive report (2026-07-16 draft), PVM-boundary findings 1–3, 5–9, 11, 13–14, 16–17, 24–25, 32, and design-flaw §F.

Relationship to the proposal. This register supersedes nothing. It is the evidence base; `pvm-interpreter-isolation-proposal.pdf` is the architecture decision. §5 bucket 3 is the only place they touch.

Unmerged branches. Two claims depend on code that is not on devnet, and both say so at the point of use: PR #1106 head eeeb0b5f for the `id()` fix location, and PR #1105 head 4f96ded6 for the differential harness, whose `examples/differential_ci.rs` does not exist on devnet at all.

“Fixed” means “proposed in an open PR” unless a merge sha is given. None of #1105, #1106, or #1110 is merged as of 2026-07-31; all three target devnet.

Path convention. Several basenames are ambiguous in the tree — `lib.rs` matches 35 paths, `mod.rs` 20 — so anchors carry enough path prefix to disambiguate. Two collisions are worth naming explicitly because they are easy to conflate: `vm/src/vm/thread.rs` is `async/coroutine` thread-local state, while `vm/src/stdlib/thread.rs` is the `_thread` module; and `vm/src/signal.rs` is distinct from `vm/src/stdlib/signal.rs`, which is the one that matters for §2.2.

6.1 Status of the Trail of Bits findings

Re-checked at 2b2e73ab: every ticket below has commits reachable from the pin, and both shas named are ancestors.

Finding	Sev	Status	Fix
1 — nested <code>os import</code> bypasses deploy gate	High	FIXED + test	COW-2418
7 — subscribers steal tokens from emitters	High	FIXED + test	COW-2424 00478bb5
11 — frozen accounts transfer via PVM	High	FIXED , fail-closed	COW-2428 e7a4180f
16 — mutable import-guard policy	High	CLOSED (freeze barrier)	COW-2434
17 — warm interpreter leaks state	High	PARTIAL — §§3, 4	COW-2434
3 — silent skip of failed token ops	Med	PARTIAL (call-time validation)	COW-2420
13 — <code>transfer_from</code> consumes allowance	Low	FIXED + test	COW-2430
14 — <code>initial_supply > max_supply</code>	Low	FIXED + test	COW-2431

The honest readout: most of the audit findings were closed quickly and well — six fully, two partially — and the audit was also right about the architecture. Finding 17 is where both hold at once. COW-2434 closed the object-mutation half of warm-interpreter reuse; the remaining half is the Rust-side and native surface this register maps.

6.2 The differential harness, and two things that would make it say what it appears to say `examples/differential_ci.rs` plus `isolation_corpus/`, on the #1105 branch, checks the one invariant every design must satisfy — **execution N+1 must not depend on execution N** — by comparing a subject’s output and gas across cold, clean-warm, and adversarially primed regimes. It is the right instrument, and it is the natural home for the evidence behind an audit response.

It drives `execute_tx_with_options` with a minimal host rather than the production `execute_handler`, so today it is a leak **ratchet**, not a production acceptance test. Two refinements would close the gap between what it asserts and what it appears to assert:

- **Sharpen what each row asserts.** Only three subjects observe anything — `len(sys.modules)` plus `gettrace`, `id()`, and `deterministic random` — so an adversarial setup only *asserts* a channel that moves one of those three. `settrace` and the `id()` divergence are genuinely asserted; most of the rest are not. Worse, several setups perform a lazy import that bumps `len(sys.modules)` and turns the row red *for the import rather than for the named channel*, a false attribution. Net: roughly 2–3 channels asserted, not 8. Adding one channel-specific subject per setup fixes this.
- **The options are the masking configuration, not production.** `build_options` starts from `ExecutionOptions::default()` with `..DeterminismOptions::default()` (`examples/differential_ci.rs:243-256` on `4f96ded6`). That default blacklist denies `threading`, `asyncio`, `socket`, `ssl`, `subprocess`, `signal`, and `select` — several of which production whitelists — and `continuation: Some(Fsm)` disables `checkpoint_exit`, which is precisely the configuration that masks §2.1. Two edits fix it: `DeterminismOptions::actor_execution()` and `continuation: None`.

Separately, the checkpoint kill (§2.1) needs its own focused production-config regression — the harness masks it (its options force `checkpoint_exit=false`). The `_signal` call (§2.2) also warrants a regression, though on-chain it did not kill; the harness runs subjects in-process, so it could not safely exercise a process-killing payload in any case.

6.3 On-chain empirical validation

Because source inspection can establish a *capability* without establishing that a deployed actor can *reach* it, the load-bearing findings were tested end-to-end on a live single-validator network (river, node `v0.0.34-b5b407d`) by deploying and executing real actors. This surfaced a control invisible to source reading — the deploy-time import resolver (§1) — and it reclassified several findings. Each live channel below carries one of three verdicts:

- **confirmed on-chain** — deployed, executed, and the escape observed;
- **deploy-blocked** — the capability exists in the interpreter but a deployed actor cannot reach it at this pin (e.g. a direct native import the resolver rejects);
- **capability / harness-only** — reproduced in the in-process harness, on-chain reachability not yet established.

The full sweep. Every load-bearing finding was deployed and executed on `v0.0.34-0efc59f` (devnet HEAD). The governing rule, learned here: the deploy has two import gates — a top-level resolver (`extract_top_level_imports`, §1) and a full-AST deny-scan (`find_denied_import`, COW-2418). A nested import (inside the handler) evades the resolver, but the deny-scan still

rejects any nested import whose root is on the deny list. So the escape surface is exactly the **natives *not* on the deny list, imported nested**. Results:

- **Confirmed live (nested import):** `posix.system` → root command execution (`uid=0(root)`), `read /etc/shadow`, A42); `_posixsubprocess` reachable with `fork_exec` present (the auditors' exact TOB-1 module); the `sys.*` findings via a nested import `sys — orig_argv` returned the validator's real command line, `_rustpython_debugbuild=False, platform=linux, maxsize` (A12, A13, A14), and `settrace present` (A3); the nested import `sys` establishes source-reachability for the rest of the `sys.*` surface (A18 `set_int_max_str_digits`, A15 `_baserepl`) without each being individually exercised; `_socket.gethostname` (A16); `_thread.get_ident=13` (A8); `io._io.open/open_code/FileIO` file read-write (A41); `builtins.eval("6*7")=42` — real eval, COW-2228 Canceled (A44).
- **Confirmed kill:** `rustpython_checkpoint.checkpoint_bytes()` halted the validator (Restart-Count 0→1, A1, §2.1) — COW-2731 is In Progress, not merged, fires at HEAD.
- **Does not fire:** `_signal.raise_signal(SIGKILL)` — the handler survived the call, `RestartCount` unchanged (A2, §2.2). A source-verified mechanism that does not reproduce; downgraded.
- **Fixes confirmed effective (negative tests):** nested import `os/subprocess/socket/ctypes/signal/pickle/inspect/threading/select/ssl/_sre` were all **rejected at deploy** (E1101). COW-2418's full-AST deny-walk and COW-2292 work — for the names on the list.

The `sys.*` findings were reclassified by this sweep from “deploy-blocked” (true only of the *top-level* import) to **live via nested import**. The three-verdict scheme above applies to what survived; the Appendix A verdict column is the authority per row.

Version reconciliation (important). The `posix` RCE was confirmed on-chain twice: first on `v0.0.34-b5b407d` (2026-07-24), then re-confirmed after repinning the environment to a coherent current matrix whose validator ran `v0.0.34-0efc59f` — **devnet HEAD 0efc59f2** — where a deployed actor again ran `id (uid=0(root))` and `read /etc/shadow`, validator restart count unchanged. Testing only on old code can risk false positives for since-fixed bugs — but that risk does not apply to this finding, because the entire import gate is unchanged across the revisions in play. `find_denied_import` (COW-2418's full-AST deny-walk) is **byte-identical at b5b407d, the pin 2b2e73ab, and HEAD 0efc59f2** (0 lines changed; the `pvm_executor.rs` file did change between `b5b407d` and the pin, but for unrelated memory-metering, not the import scan). So COW-2418 was already present on the river node we first tested — nested imports of *denied* modules such as `subprocess` are rejected on all three, which the negative tests confirm. The `posix` RCE's three links (`posix` on no deny list; the resolver `extract_top_level_imports` top-level-only; the import-statement cache path skipping the allowlist) are likewise identical across all three, and the escape reproduces on-chain at HEAD. The register's pin `2b2e73ab` sits between the two tested revisions and shares the identical code, so the finding holds at the pinned revision. Where a result could not be transferred it is marked *harness-only* in Appendix A; the verdict column there is the authority.

6.4 Fix status against Linear — mostly residuals of closed tickets (plus one canceled, one in-progress)

The most important thing to say to the owning team: for the **Done** rows below, the fixes were written, merged, and are present at HEAD — and the findings here are what each did not

reach, verified live. The table also carries a **Canceled** ticket (COW-2228) and an **In Progress** one (COW-2731, its gating commit not yet in devnet), and one **n/a** row (`_signal`). These are not residuals of a closed fix: COW-2228 was **Canceled** — its finding (A44) is still open and unfixed, which is not the same as the ticket being open — and COW-2731 is **In Progress**, unmerged. The Done rows are the residuals; the table's row-level statuses are the authority. This is the register's thesis stated as a fact about your own backlog.

Ticket	Status	What it fixed (present at 2b2e73ab/HEAD)	The residual
COW-2284 — import os pre-cached bypass	Done	poison_precached_blacklist None-poisons cached blacklisted modules	Iterates the blacklist; posix (os's Unix name) is not on it → not poisoned → A42 root RCE
COW-2418 (TOB-1) — nested os import bypass	Done	Deny-scan walks the full AST (find_denied_imports)	Rejects only denied imports; posix is on no deny list → nested import posix passes
COW-2433 / COW-2434 (TOB-16) — mutable guard globals	Done	Froze _ALLOW/_DENY	Freeze is a thread-local; an off-barrier _thread callback still mutates them → A43
COW-2228 — builtins/__import__ re-acquisition	Canceled	—	import builtins; builtins.exec(...) reaches real exec → A44, openly unfixed
COW-2292 — whitelisted I/O modules importable	Done	Deploy scan denies socket/subprocess/... (verified on-chain: nested imports of these are rejected)	Underscore natives (_socket, _signal) and posix are not the high-level names it denies
COW-2731 — checkpoint validator halt	In Progress	PR #1110 not merged to devnet	Fires at HEAD — confirmed on-chain, RestartCount 0→1 (A1, §2.1)
— _signal.raise_signal(SIGKILL) (register's own earlier claim)	n/a	—	On-chain the SIGKILL does not terminate the validator; downgraded (A2, §2.2). Included so the correction is on record.

The pattern is uniform: each fix closes the *named* instance and leaves the *class* — which is precisely why §5 argues the durable answer is the process boundary rather than a longer list. The auditors said the same under TOB-16 (“making the lists immutable would only address this

specific mutation primitive... Python-only hardening [is] an ongoing security burden rather than a durable sandbox”).

7. Open questions for the PVM team

These are the questions whose answers change the recommendation, and they need a live box or production data rather than more source reading.

0. **Why does `_signal.raise_signal(SIGKILL)` not kill the validator?** The source path is a real `libc::raise(SIGKILL)` (§2.2), and `_signal` is reachable, yet on-chain the actor frame survived the call. If the non-firing is environment-specific (thread targeting, a masked signal, a restart faster than the check) rather than a genuine mitigation, the kill could resurface — worth understanding before treating §2.2 as closed.
 1. **Real per-block actor-execution counts on canyon.** Everything about capacity in either document is inference until this is measured.
 2. **What does the first nested `call_actor` in a transaction actually cost?** §4.1 shows the outer execution empties the pool slot, so the first nested call takes the pool-miss branch and builds a cold interpreter. If production `call_actor` transactions show no first-nested-call penalty, then the cold-build cost is smaller than assumed and part of the argument in the companion document weakens. If they do show one, it is a block-time problem that exists today, independent of any isolation work. Either answer is worth having.
 3. **Is `_ssl` or `fcntl` actually reachable to a useful end?** Both are allowlisted and denied at neither gate. Nobody has exercised them. See bucket 4.
 4. **Does the validator kernel expose soft-dirty page tracking** (`CONFIG_MEM_SOFT_DIRTY`)? This decides whether selective-restore designs are available at all.
 5. **Bare-metal x86-64 numbers.** All isolation measurements taken so far are aarch64-in-Docker, whose memory bandwidth is roughly 2–3× a typical server’s single-threaded figure. Anything restore-by-copy looks better there than it will in production.
-

Appendix A — Channel inventory

All 44 identified channels, each with a stable **ID (A1–A44)** for referring to it in review, tickets, or conversation. **Bucket 2 · 6** means bucket 2, item 6. Anchors are given in short form to fit; expand with the path key below and resolve with `git show 2b2e73ab:<path>`.

Path key. `vm/...` = `pvm/crates/vm/src/...` · `stdlib/...` = `pvm/crates/stdlib/src/...` · `runtime/...` = `pvm/crates/pvm-runtime/src/...` · `execution/...` = `execution/src/...` · `validator/...` = `validator/src/...` · `rpc/...` = `rpc/src/...` · `Lib/...` = `pvm/Lib/...`. Where a basename is ambiguous the tree path disambiguates: `vm/vm/thread.rs` is coroutine and async-generator thread-local state, `vm/stdlib/thread.rs` is the `_thread` module.

ID	Face	Channel	Anchor	Status	Bucket
A1	kill	rustpython_checkpoint.checkpoint*	vm/frame.rs:516, :595, :2809-2827	live, confirmed on-chain (kill; RestartCount 0→1; COW-2731 not fixed)	1
A2	kill	_signal.raise_signal(SIGKILL)	vm/stdlib/signal.rs:119,449,470	reachable but does NOT kill on-chain — down- graded, §2.2	1
A3	VM fields	sys.settrace	vm/stdlib/sys.rs:796, :794	settrace reachable on- chain (nested import sys); cross-actor <i>firing</i> via an un- committed harness (§4.1)	2 · 8
A4	VM fields	sys.setprofile	vm/stdlib/sys.rs:769-772	live, source-verified (set- trace confirmed on-chain; same mechanism)	2 · 8
A5	VM fields	pool key omits actor identity	runtime/lib.rs:486-503	source-verified	3
A6	VM fields	intra-tx sibling reuse	runtime/lib.rs:606-629, :703-715	source-verified	3
A7	_thread	start_new_thread barrier bypass	vm/sandbox.rs:27, :104-119; vm/stdlib/thread.rs:298-327	live, source-verified	2 · 7
A8	_thread	get_ident	vm/stdlib/thread.rs:265, :317-325	live, confirmed on-chain (get_ident=13)	2 · 7
A9	_thread	_set_sentinel	vm/stdlib/thread.rs:360-367, :341	probe-observed	2 · 7
A10	consensus	id() cold vs. warm	runtime/lib.rs:753, :761-791	live, runtime-reproduced	1
A11	consensus	id() address reuse	vm/object/det_id.rs:35,44-54	unassessed	4
A12	consensus	sys.orig_argv	vm/stdlib/sys.rs:210-211	live, confirmed on-chain (nested import; got valida- tor cmdline)	2 · 6
A13	consensus	sys._rustpython_debugbuild	vm/stdlib/sys.rs:52; determinism.rs:236 vs :128	runtime/ live, confirmed on-chain (=False)	2 · 6

ID	Face	Channel	Anchor	Status	Bucket
A14	consensus	sys.platform, maxsize, target triple	vm/stdlib/sys.rs:70, :66, :43, :47-48	live, confirmed on-chain (platform=linux, nested import)	2 · 6
A15	consensus	sys._baserepl	vm/stdlib/sys.rs:279, reads stdin at :280-286	unassessed	4
A16	consensus	_socket.gethostname	stdlib/socket.rs:2584	live, confirmed on-chain (hostname)	2 · 5
A17	consensus	math / float ** → platform libm	runtime/determinism.rs:172; pvm/Cargo.toml:249	reachability verified; im- pact unassessed	2 · 10
A18	consensus	sys.set_int_max_str_digits	vm/vm/mod.rs:106; vm/stdlib/sys.rs:752-755	reachable via nested import sys (sys confirmed reachable on-chain; this at- tribute not individually ex- ercised)	2 · 6
A19	consensus	_socket.setdefaulttimeout	stdlib/ socket.rs:3324 (DEFAULT_TIMEOUT), :3333-3334	probe-observed	2 · 5
A20	host escape	_posixsubprocess.fork_exec	stdlib/posixsubprocess.rs:35, 48, :380; stdlib/lib.rs:193	live, source-verified	2 · 5
A21	host escape	_socket egress	stdlib/socket.rs:1533, :1319; stdlib/ lib.rs:168	live, source-verified	2 · 5
A22	host escape	io.FileIO	vm/stdlib/io.rs:4738, :4829	live, source-verified	2 · 6
A41	host escape	io._io.open / io.open_code / io.FileIO — host file read/write via the allowlisted io wrapper (§2.3)	Lib/io.py:52, :55-56; vm/stdlib/ io.rs:4387-4393, :4397-4399, :4829	live, confirmed on-chain	1
A42	command exec	nested import posix; posix.system(...) — root RCE; posix unlisted, resolver top-level-only, cache skips allowlist (§2.3)	vm/import.rs:38; vm/stdlib/posix.rs:23; vm/stdlib/os.rs:1243-1245; execution/ actor_instruction.rs:217-218; vm/vm/ mod.rs:851-860	live, confirmed on-chain (root)	1
A43	import-pol- icy bypass	_ALLOW.add from an off-barrier _thread callback widens the allowlist	guard.rs:14-23; vm/sandbox.rs:27; vm/stdlib/ thread.rs:298-327	harness-reproduced	1

ID	Face	Channel	Anchor	Status	Bucket
A44	code exec	import builtins; builtins.exec/ compile — reflection re-acquisition (COW-2228, Canceled)	Lib reflection guard; COW-2228	live, confirmed on-chain (eval("6*7")=42; COW-2228 Canceled)	1
A23	native	_ssl	runtime/determinism.rs:62, :128	unassessed	4
A24	native	fcntl (note: _fcntl is on the allowlist but no such module exists)	runtime/determinism.rs:104, :105; stdlib/ lib.rs:159	unassessed	4
A25	native	underscore aliases evade AST scan	execution/pvm_executor.rs:414-427	source-verified	2 · 5
A26	stdlib state	sys.set_asyncgen_hooks	vm/vm/thread.rs:13-14	probe-observed	2 · 9
A27	stdlib state	sys.set_coroutine_origin_tracking_depth	vm/vm/thread.rs:12	probe-observed	2 · 9
A28	stdlib state	atexit.register	module-level	probe-observed	2 · 6
A29	stdlib state	codecs.register / register_error	module-level	probe-observed	2 · 6
A30	stdlib state	copy.deepcopy _nil=[] default	vm/sandbox.rs:382-385	probe-observed	3
A31	stdlib state	closure-cell replacement	vm/sandbox.rs:373-375 (skipped deliberately)	probe-observed	3
A32	stdlib state	_abc_registry WeakSet	vm/sandbox.rs:382-385	probe-observed	3
A33	stdlib state	property descriptor	vm/sandbox.rs:397-403	probe-observed	3
A34	stdlib state	lazy-import sys.modules persistence	vm/vm/mod.rs:821-890, :65-82	source-verified	3
A35	stdlib state	write_bytecode defaults true	vm/vm/setting.rs:240	unassessed	4
A36	freeze bypass	StoreGlobal skips writable()	vm/builtins/dict.rs:645-655 vs :274-283, :281; Lib/threading.py:72-80	source-verified	2 · 8
A37	liveness	no catch boundary around actor exec	validator/main.rs:342; rpc/rpc.rs:747, :367-376	source-verified	3
A38	liveness	metering wired but not enforcing	execution/execution/transaction.rs:616-619; execution/basefee.rs:119,244-249; vm/ frame.rs:455-467	source-verified	3

ID	Face	Channel	Anchor	Status	Bucket
A39	atomicity	≥10 host paths bypass rollback()	execution/ pvm_host.rs:1304-1328, :1520-1600, :1542; execution/stream_key_manager.rs:1067,1078	source-verified (floor)	1
A40	atomicity	effect list ≠ what a tx committed	execution/pvm_executor.rs:54-67	source-verified	1

Citing an issue. Use the ID: “A13 is live on devnet” is unambiguous, whereas “the sys finding” is not. IDs are stable for this revision of the register; if a row is struck because the finding is fixed, the ID is retired rather than reused.

Entries in bucket 3 are the population §5 bucket 3 is about: no per-channel patch in buckets 1 or 2 is proposed for them. Entries in bucket 4 are reachable on inspection but have not been exercised, and are deliberately not counted among the eleven live channels in §3.3.

Appendix B — Ready-to-file tickets

Nothing here has been filed. These are drafts, so that agreeing costs one pass through Linear rather than a writing session. Disagreeing costs nothing. Where a PR already exists, the ticket references it rather than restating the work.

Severities use the same vocabulary as the register: a **validator-kill** stops the process, a **consensus fork** makes validators disagree, a **cross-actor** issue lets one actor affect another, and a **host escape** reaches outside the interpreter.

From bucket 1 — fix now

B1. Land the rustpython_checkpoint gate (COW-2731) *Severity: validator-kill. Closes §2.1. Not superseded by any boundary.* PR #1110 is open and not merged. Acceptance: an actor calling `checkpoint_bytes()` under production options (`continuation: None`) no longer terminates the validator; the `checkpoint_gate` regression suite passes; the continuation suite stays green. Add a production-config regression that would have caught this — the existing corpus runs with `continuation: Some(...)`, which forces `checkpoint_exit = false` and masks it.

B2. Deny `_signal` to actor frames (part of the native-module class) *Severity: reachable native; the claimed SIGKILL kill does not reproduce on-chain (§2.2). Folds into B5.* `_signal` is reachable via nested import (`SIGKILL=9`, `raise_signal` present, confirmed on-chain), but `raise_signal(SIGKILL)` did **not** terminate the validator in testing — so this is not a standalone validator-kill ticket. Deny `_signal` as part of the native-module default-deny (B5). Acceptance: `import _signal` fails closed in actor code; a regression asserts it. Separately, an open question worth a note (§7): *why* the SIGKILL does not kill, in case the non-firing is environment-specific rather than a real mitigation.

B3. Land the `id()` cold/warm determinism fix (COW-2291 follow-up) *Severity: consensus fork. Closes the counter-offset half of §4.3.* PR #1106 is open and not merged, head `eeeb0b5f`. It changes actor-visible `id()/repr()` values, so it is a **state-transition-function change** and must ship in a coordinated genesis-replacement, not an in-place patch. Acceptance: identical actor source returns identical `id()` on cold and warm interpreters; nested `call_actor` is unaffected. Note in the ticket that address reuse (below) is explicitly *not* covered, so the fix is not recorded as closing the whole class.

B4. Add a mechanically-checked store-write choke point *Severity: atomicity. Closes §4.7. Required under any architecture.* At least ten host paths write the store during execution and bypass `rollback()`, which clears staged in-memory queues only. Acceptance: every store write during actor execution routes through one enforced path; the exact set is produced by the

check rather than by reading; each path that intentionally commits outside per-tx rollback (as `econ_intent_gate` does, for a documented and sound reason) is explicitly marked as such rather than implicitly so.

B4b. Close the import-boundary escape class (posix command execution, native `_io`, the `io` wrapper, and `_ALLOW` mutation) *Severity: critical — arbitrary command execution. Closes A41, A42, A43 (§2.3, §4.4). The allowlist-keying and mutation halves are ordering/enforcement defects, not superseded by the boundary; the wrapper-reference half is.* Four changes, each closing a distinct hole; all are wanted, because each leaves the others open:

1. **Key the cache mitigation off the allowlist, not the blacklist.** `poison_precached_blacklist` poisons only blacklisted names, so every preloaded module absent from the allowlist (`posix`, `_io`, `marshal`, `_imp`, `_frozen_importlib`) stays live. Poison-or-deny every resident module not on the allowlist. *Acceptance:* `import posix`, `import _io`, `import marshal` each fail closed under production config, with a regression per module and a test that a newly-preloaded module is denied by default rather than by omission.
2. **Neutralize the `io` wrapper's native references.** `pvm/Lib/io.py` retains native `_io` as `io._io` and copies `open_code/FileIO`, none of which the guard rebinds. *Acceptance:* `io._io.open(path)`, `io.open_code(path)`, and `io.FileIO(path)` all fail closed in actor code.
3. **Freeze the import policy against the `_thread` bypass.** `_ALLOW` is reachable as `__import__.__globals__["_ALLOW"]` and mutable from an off-barrier thread (§4.2). Either enforce the barrier on spawned threads (ties into ticket 7 / A7) or hold the policy in Rust rather than a Python set. *Acceptance:* an `_ALLOW.add()` from a spawned thread does not make a non-allowlisted module importable.
4. **(Preferred, subsumes 1.)** Run the positive allowlist check against a `sys.modules` hit, not only on a cache miss — preserving COW-2433's deny-check-ahead-of-cache ordering.

Doing only the obvious `_io` patch (deny `import _io`) closes none of the others: `posix` still executes commands, and the `io` wrapper still opens files. This is why it is filed as a class.

From bucket 2 — bridge controls

Each of these says plainly that it is superseded if a process boundary lands, so nobody is surprised later.

B5. Default-deny the native OS-module class at runtime *Severity: host escape (the `_signal` process-kill in this class did not reproduce on-chain, §2.2). Closes §3.2 and much of §4.4. Superseded by a boundary.* `_signal`, `_posixsubprocess`, `_socket`, `_ssl`, `fcntl` are on the production whitelist and omitted from both deploy deny arrays, so a nested import reaches them. The high-level names (`socket`, `subprocess`, `signal`, `ssl`) are denied by the deploy scan even when nested — but they remain reachable via a **dynamic** `__import__("socket")`, which the scan (reading static import statements) does not see and which the runtime guard admits (they are on `_ALLOW`, absent from `_DENY`; §3.2). Must be enforced at **runtime**, not by extending the deploy AST scan: a dynamic `__import__` evades that scan, and the underscore aliases were never on it. Must also cover modules already imported by the preamble. *Acceptance:* each named module is unreachable from actor code by direct import, dynamic import, and via an already-loaded reference.

B6. Positively construct a minimal actor environment *Severity: cross-actor + consensus fork. Closes §3.1 and §3.3 broadly. Superseded by a boundary.* Replace the stock-environment-minus-blocklist model with a default-deny allowlist of builtins, globals, and modules. This is the control that changes the completeness burden from per-name to enumerable. Acceptance: an actor sees only names on the allowlist; adding a name is a deliberate, reviewed act; `io.FileIO`, `sys.orig_argv`, and `sys._baserepl` are absent.

B7. Gate the whole actor-visible `_thread` API *Severity: cross-actor mutation + consensus fork. Closes §4.2 and part of §4.3. Superseded by a boundary.* Denying thread creation alone is insufficient — `get_ident` needs no spawn and is directly returnable. Acceptance: `get_ident`, `start_new_thread` and its return value, timed locks, `_count`, `interrupt_main`, and `stacksize` are each denied or pinned to a deterministic value; a regression covers `get_ident` specifically, since it is the one a creation-only fix would miss.

B8. Neutralize `sys.settrace` / `setprofile`, and audit the `StoreGlobal` path *Severity: cross-actor code execution. Closes §4.1. Superseded by a boundary.* Two parts, and the second is the one that gets forgotten: `trace_func/profile_func` are pub fields that a per-execution reset closes, but `threading.settrace()` writes `_trace_hook` through the Rust `PyDict::set_item` path used by `StoreGlobal`, which skips the `writable()` check the Python `__setitem__` enforces (§4.5). Acceptance: both the direct and the `StoreGlobal` route are covered, with a regression for each.

B9. Scoped `asyncio` thread-local save/install/restore per execution *Severity: cross-actor. Closes the `asyncgen/coroutine` rows of §4.5. Superseded by a boundary.* A save/restore rather than a denial: `async-generator` hooks and `coroutine-origin` tracking are load-bearing for `asyncio`, so denying the setters would break a direct-`asyncio` actor. The on-chain SDK deliberately avoids `asyncio` (COW-2331), so the exposure is a direct-`asyncio` actor. Acceptance: hooks set by one execution are not observable by the next; a direct-`asyncio` actor still runs.

B9b. Decide the float story — deny `math` and `float **`, or pin the target *Severity: consensus fork if reachable. Closes the `float` row of §4.3. Superseded by a boundary only in part — a boundary isolates the interpreter, it does not make `libm` agree across machines.* `math` is on the actor whitelist and `float **` lowers to the host `pow`; `math` delegates to `pymath`, a C-ABI shim over the system `math` library. Two viable answers: deny `math` and `float **` to actor code, or constrain the build so the `math` path does not reach the system `libm`. Acceptance: either actors cannot reach a platform-dependent float operation, or the set of operations they can reach is enumerated and shown to be bit-identical across every target we build. Note that this is the one item on the list that may need a measurement before the decision can be made — see B15.

B9c. Set `write_bytecode = false` *Severity: unassessed. Closes the `.pyc` row of §4.5. Superseded by a boundary.* `write_bytecode` defaults `true` (`vm/src/vm/setting.rs:240`) and nothing in `pvm-runtime/` or `execution/` overrides it, so the interpreter is configured to write `.pyc` files and to prefer a matching one over recompiling. Acceptance: the setting is explicit in the production execution options, with a test asserting it, so it cannot drift back with an upstream default change.

Tooling and evidence

B10. Make the differential harness assert what it appears to assert *Severity: test correctness. Supports §6.2.* Only three subjects observe anything, so most adversarial setups assert nothing about their named channel, and several turn a row red for a lazy `import bumping len(sys.modules)` rather than for the channel — a false attribution. Net: roughly 2–3 channels asserted, not 8. Acceptance: one channel-specific subject per setup; each row fails for its own channel and no other.

B11. Point the differential harness at the production configuration *Severity: test correctness. Supports §6.2 and B1.* `build_options` starts from `ExecutionOptions::default()` with `..DeterminismOptions::default()`, whose `blacklist` denies modules production whitelists, and whose `continuation: Some(Fsm)` disables `checkpoint_exit` — the exact configuration that masks §2.1. Two edits: `DeterminismOptions::actor_execution()` and `continuation: None`. Acceptance: the harness runs under production determinism options; a checkpoint-kill subject would fail before B1 lands.

B12. Commit a production-path regression for the settrace cross-actor result *Severity: evidence gap. Supports §4.1.* The mechanism is source-verified, but the runtime result was produced with an uncommitted `execute_handler` harness, while the committed corpus goes through the `pvm-runtime` path — so the result is not currently reproducible from the tree. Acceptance: a committed test drives the production `PvmExecutor::execute_handler` with two actor addresses and asserts that the first actor’s callback does not run inside the second.

Assessment

B13. Assess `_ssl` and `fcntl`, and drop the dead `_fcntl` allowlist entry *Severity: unknown by construction. Closes bucket 4.* Both are on the production import allowlist and denied at neither gate, so they are reachable on inspection, but neither has been exercised. They are deliberately excluded from the eleven live channels in §3.3 for that reason. Acceptance: each is either demonstrated reachable to a concrete end and moved into the live set, or shown inert and recorded as such. Either outcome is worth having; the current state is the only one that is not.

Separately and cheaply: the allowlist entry `"_fcntl"` (`pvm-runtime/src/determinism.rs:105`) names no module — the registration is `"fcntl"` (`stdlib/src/lib.rs:159`) — so it grants nothing and should be removed. Worth a moment’s thought as to how it got there and whether any other entry is similarly inert, since an allowlist nobody audits is the thing §3 is about.

B15. Measure whether actor-reachable float operations actually diverge *Severity: informs B9b. Closes the impact half of the float row in §4.3.* Reachability is source-verified; impact is not. Run the actor-reachable math surface and `float **` over a fixed input set on each target we build for, and compare bit patterns. Acceptance: either a concrete divergence, which makes B9b urgent and settles which operations matter, or a clean result across targets, which lets B9b be scoped down to a documented constraint on the build rather than a denial. This is cheap and it is the gate on a decision, so it is worth doing before arguing about the remedy.

B14. Determine whether `id()` address reuse is reachable *Severity: consensus fork if reachable. Closes the second half of §4.3.* `EXEC_OBJECT_IDS` is keyed on the raw heap pointer, so B3 fixes

the counter offset but not pointer reuse: `a=C(); i=id(a); del a; b=C(); return i==id(b)` may still differ cold versus warm, because a warm interpreter's allocator free-lists are fragmented differently. Acceptance: a probe either demonstrates divergence — in which case this is a live consensus fork and B3's ticket is corrected not to claim the class — or shows it unreachable, and that is recorded.